

Problem J1. Conveyor Belt Sushi

The total cost is obtained by multiplying the number of plates of each colour by its price and summing:

$$3R + 4G + 5B.$$

Then print this value.

Complexity. The running time and memory usage are $O(1)$.

Problem J2. Dusa And The Yobis

The number of Yobis is not given in advance, so a `while` loop is appropriate. Read Dusa's starting size. Then repeatedly read the next Yobi. If the Yobi is smaller than Dusa, Dusa eats it and its size increases by the Yobi's size. Otherwise, Dusa stops and the current size is printed.

The loop condition is driven by the input values, not by a fixed count. This is the main twist compared with earlier Junior problems.

Complexity. The running time is linear in the number of Yobis read, and the memory usage is constant.

Problem J3. Bronze Count

We need the third-highest distinct score, called the bronze score, and the number of participants with that score. One simple method is to sort all scores, then scan from largest to smallest while counting how many distinct score values have been seen. When the third distinct value is reached, count how many times it occurs.

It is also possible to solve the problem in a constant number of passes without sorting by maintaining the best three distinct scores and their counts.

Complexity. Sorting gives $O(N \log N)$ time. The one-pass variant uses $O(N)$ time and $O(1)$ extra memory.

Problem J4. Troublesome Keys

Compare the intended key presses with the displayed string. The silly key produces the wrong displayed letter, while the quiet key produces no displayed letter. If the lengths differ, the quiet key was pressed at least once.

One robust method is to locate the first mismatch while scanning the two strings. This reveals candidates for the silly key and the wrong letter. Continue scanning, accounting for the effect of the silly key, until a mismatch that cannot be explained by the silly key identifies the quiet key. Another method is to find candidate letters using frequency counts and then simulate the keyboard with the two possible assignments.

For the largest input sizes, avoid repeated searches through the strings; use a single pass or a fixed-size frequency table for the alphabet.

Complexity. The solution runs in $O(N + \Sigma)$ time, where Σ is the alphabet size.

Problem J5. Harvest Waterloo

The task is to find every cell reachable from the starting position without crossing hay. Store the patch as a two-dimensional array. Then perform a flood fill, either BFS with a queue or DFS with a stack.

When a cell is first visited, add the value of its pumpkin to the answer: small pumpkins contribute 1, medium pumpkins 5, and large pumpkins 10. From that cell, try the four neighbouring cells. A neighbour is added

only if it is inside the grid, is not hay, and has not already been visited. Marking visited cells immediately prevents duplicate work.

Recursive DFS is also possible, but languages with small recursion limits may require an explicit stack or an increased recursion limit.

Complexity. Each cell is processed at most once, so the time and memory usage are $O(RC)$.